

University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

Project Specifications and Risk Assessment
Podcasts Application with Automatic Advertisement Skipping and Recommendation Engine

Group 2017.001

Prepared by

Consultant: Prof. Stephen Smith

May 30, 2016

Table of Contents

1	High Level Description	3
1.1	Motivation	3
1.2	Project Objective	3
1.3	Block Diagram	3
1.3.1	iOS Application Subsystem	3
1.3.2	Web Application Subsystem	5
1.3.3	Backend Server Subsystem	5
1.3.4	Recommendation Subsystem	5
2	Project Specifications	6
2.1	Functional Specifications	6
2.2	Non-functional Specifications	8
3	Risk Assessment	9
4	References	10

1 High Level Description

1.1 Motivation

There were 60 000 active podcasts on the US iTunes store in 2015, which is an exponential increase from around 10 000 podcasts in 2008 [1]. Meanwhile, the percentage of Americans consuming podcasts increased linearly from 9% to 17% in the same time frame [2]. This data points to a peculiar scenario where the podcast market is growing faster in supply than it is in demand. We believe the reason for this situation is that existing podcast clients use outdated technologies that are not optimized for mobile devices and that do not provide their users with interesting services based the data they collect from them. LibSyn, one of the largest podcast hosting companies, reports that of the 2.6 billion podcasts downloaded from them in 2014, 63% were requested from mobile devices which is up from 43% in 2012 [2]. With the increasing prevalence of mobile devices and the stagnant feature set of current offerings, we believe that podcasting clients are ripe for technical advancement and improvement.

1.2 Project Objective

The objective of this project is to design a podcast client that makes consumption more appealing to users by incorporating features such as advertisement-skipping, smart recommendations and better shareability of podcasts on social media. Current podcast clients are lacking in these types of features which we believe digital-age consumers have grown to expect from media providers.

1.3 Block Diagram

The proposed solution is a podcast client that has a dedicated backend service capable of generating recommendations and inferring advertisement locations based on crowdsourced user input. The architecture is shown in **Figure 1** and is composed of four main subsystems: The iOS (iPhone operating system) application subsystem, the web application subsystem, the backend server subsystem and the recommendations subsystem.

1.3.1 iOS Application Subsystem

The iOS Application is broken down into five major components that are logically separate. The first component is the networking layer which facilitates the communication between the client itself, the backend server subsystem and Apple iTunes podcast database. The data received from the networking layer is managed through the controller subsystem. Depending on the data being supplied, the controller will persist the data locally through the persistent storage component, provide a data source to the audio playback component or provide information for the user interface. The persistent storage component manages fetching and storing audio files, the associated metadata, user interaction statistics as well as user information. The audio playback component outputs the podcast audio to the user when supplied a data stream from the controller component. The user interface takes inputs from the user and outputs the corresponding visual response.

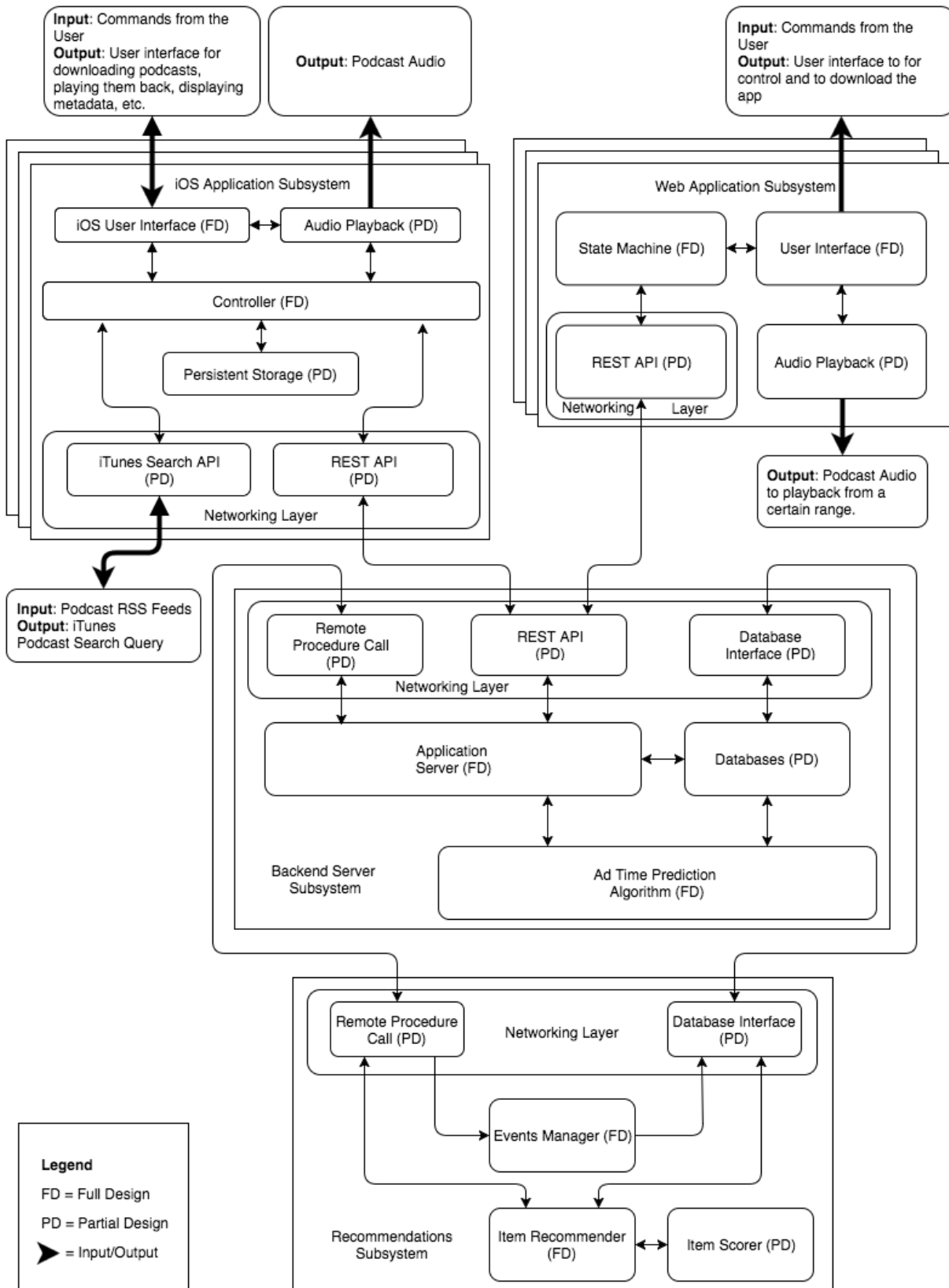


Figure 1: A block diagram showing the software components of the application

1.3.2 Web Application Subsystem

The web application contains four subsystems that are architecturally separated by their use case. The first subsystem is the network layer which communicates with the backend server through the use of a REST API. REST APIs are application program interfaces that expose a set of functionality useful for building out client frontends that interact with web service backends [3]. Since the main purpose of the web application is to play back a certain range of a podcast in the user's web browser, the data being passed will contain a feed to the audio file in question and a timestamp for the playback period. The state machine will have to save this information while the session is open. Finally the user interface will allow the user to view the metadata associated with the audio file being sent and play/stop the audio. The audio playback portion of the web application mirrors that of the iOS and allows the user to navigate the audio file. Lastly the web app will also include an embedded link in the user interface to download the iOS app.

1.3.3 Backend Server Subsystem

The backend server contains four major components that are logically separate. All communication with external systems is handled by the networking layer which consists of three sub-components; the Remote Procedure Call (RPC) interface, the REST API and the Database interface. A Remote Procedure Call (RPC) is a technique for making a function call from one software system to another over a network [4]. The Application Server holds the business logic of the RPC and API calls and directly interacts with the database to manage data. The Database Interface is used exclusively for external access to the database from the recommendation subsystem. The last component is the Ad Time Prediction Algorithm which will be an algorithm used by the Application Server to infer the advertisement times of a given podcast episode based on collected user data that is present in the database.

1.3.4 Recommendations Subsystem

The recommendations system contains four different components that are logically separate. Almost all interactions with the recommendations system must go through the networking layer, which is itself composed of the RPC sub-component and the Database Interface sub-component. The Events Manager component is an important part that serializes, categorizes and stores the events to persistent storage such as a database. Due to the fact that the events need to be stored in the database, this part is very closely coupled with the Database Interfacing sub-component. Lastly, the Item Recommender and Item Scorer are two related components that also work together. The Item Recommender packages, sorts and caches recommendation results. The Item Scorer is responsible for computing the recommendation score for every podcast with relation to the user and returns the top recommended podcasts using the data provided by the overall subsystem [5].

2 Project Specifications

The project specifications can be divided into functional specifications and non-functional specifications. Each specification is categorized as either essential or nonessential based on whether or not it is required for our project to succeed.

2.1 Functional Specifications

Ten functional specifications were identified and can be found in **Table 1**.

Table 1: Functional Specifications

Subsystem	Specification	Additional Details	Necessity
iOS Application	iOS application must be able to automatically skip over portions of the podcast that the server component has deemed to be advertisements.	In order to verify that this subsystem is working properly, automated test cases will be added to this block. These tests will be able to verify that the application does not play back portions of the podcast that are marked as advertisements.	Essential
iOS Application	iOS application must be able to playback locally downloaded and streaming podcasts. When streaming, there must be no buffering on networks with at least 1 Mbps of downlink capacity.	These are standard playback features that are present in competitive products. To verify the streaming performance, we can use the network degradation utilities built into the iOS Simulator.	Essential
iOS Application	Statistics that iOS Application must track: The time ranges of the podcast that the user skipped over and the timestamp at which the user stops listening to the podcast entirely.	This data must be collected on the iOS application for the Ad Time Prediction Algorithm component to work.	Essential
iOS Application	The iOS application should be simultaneously download or stream at least 2 different MP3 files from a remote URL while on Wi-Fi.	The app allows for the user to download podcasts to listen to offline. Simultaneous download capabilities provide a better user experience.	Non-Essential

Backend Server	Backend Ad Time Prediction Algorithm must interpret user actions given to the server to determine the advertisement ranges within the podcast, accurate to within ± 5 seconds for podcasts that have inferrable data from at least 10 different users.	Inferrable data includes skipping forward / fast-forwarding when an advertisement starts playing, explicitly marking advertisement ranges using the user interface, or answering questions about whether or not they just heard an advertisement.	Essential
Backend Server	The user database must be able to hold up to 5000 user's information. The podcast database must be able to hold podcast metadata for up to 10 000 podcasts.	This must account for the ad skipping data, user podcast recommendations, and collected metadata on all podcasts.	Essential
Backend Server / Web Application	The backend must be capable of generating a URL link which can be shared with various social media platforms such as Facebook Messenger or Slack. The URL must be live for at least a week and must direct the user to an interface where they can listen to a small clip of a podcast.	This allows users to listen to small clips of podcasts without having the mobile application. They can also visit the site to listen to podcasts and to download the app.	Non-Essential
Recommendations	Must provide at least 3 recommendations to a specific user based on existing user data.	The recommendations are returned in an unordered set to not impose artificial ranking.	Essential
Recommendations	The Recommendations Subsystem must be able to generate recommendations in under 25 seconds per user.	This is permissible because the recommendations are not generated on demand but instead are computed and refreshed on a regular schedule.	Essential
Recommendations	The system must generate recommendations for every user once a day if the user data has changed.	The user data would include any new podcasts a user has listened to or subscribed to.	Non-Essential

2.2 Non-functional Specifications

Four non-functional specifications were identified and can be found in **Table 2**.

Table 2: Non-functional Specifications

Subsystem	Specification	Additional Details	Necessity
Overall System	The system should have strong computer security.	User authentication should happen in a simple and secure manner. Users should only be able to access data that they have proper authorization to view. The backend infrastructure and database tables should only be accessible by the product engineers.	Essential
Backend Server	System should be resilient to malicious behaviour. Users should not be able to spam the servers with requests.	Unauthorized use of our API could cause the system to go down due to overloading the server.	Non-Essential
iOS Application	The application should have low battery and data usage. It should perform as well as existing podcast clients and should not to be a burden on the hardware.	The app collects a large amount of data in order to better serve the user. This should not affect the battery life severely. The application will wait until the phone is charging and on WiFi to do computationally expensive tasks.	Non-Essential
Recommendations	Results of the partial computations will be cached to provide a significant performance boost.	The recommendation engine instance for a given user can be serialized to a file so that it need not be instantiated again for every user call.	Non-Essential

3 Risk Assessment

The major risks identified are detailed in **Table 3** and include the cold start risk, team coherence and scheduling risks, and interoperability risks.

Table 3: Risk Assessment

<u>Cold Start Problem</u> There exists a technical risk with the recommendations engine and the ad time prediction algorithm with regards to the initial amount of user data. The recommendation engine is driven by collaborative filtering algorithms and suffers from the traditional cold start problem if there is an initial lack of data. The ad time prediction algorithm also depends on user data in order to accurately determine the locations of advertisements within a podcast. Probability: High, Impact: High Solution: <i>To reduce the likelihood of this risk</i> we propose an alternate solution which is to initially curate categories and handpick podcasts to recommend. During this time, we can actively collect data from our users in order to start building out our learning capabilities for recommendations and ad skipping. <i>To reduce the impact of this risk</i>, we can ensure that our data collection and analysis algorithms are correct and show that they would function properly given a sufficient amount of user data.
<u>Team Coherence and Scheduling</u> Since the project is technically complex and requires coordination between five people, there is a modest chance that there will be scheduling conflicts between the group members. Additionally, the members of this group have not worked together on another team project prior to this. Probability: Medium, Impact: Medium Solution: A team calendar will be used to synchronize meeting times between the different group members. Two regular meetings that were 1 hour long were scheduled every week. The purpose of these meetings will be to sync up on progress and make sure that all members of the team are on track. Additionally, a meeting has been set up with Prof. Smith every other week to obtain regular feedback on the project.
<u>Interoperability</u> Since the project involves four separate subsystems, there is a risk that there will be conflicts with ensuring that each subsystem properly communicates with one another. This is due to the fact that each subsystem is potentially running on a separate physical machine and may be written in completely different programming languages. Probability: Low, Impact: High Solution: <i>To reduce the likelihood of this risk</i>, our solution is to have well defined interfaces in place before beginning to work on each component. All members of the group are currently enrolled in ECE 454, Distributed Systems, and will be using techniques taught in that class in order to build out components in a composable way. <i>To reduce the impact of this risk</i>, we will be regularly testing the interoperability between different components so that we can catch and fix problems early on.

4 References

- [1] N. Vogt, "Podcasting: Fact Sheet", *Pew Research Center's Journalism Project*, 2015. [Online]. Available: <http://www.journalism.org/2015/04/29/podcasting-fact-sheet/>. [Accessed: 29- May- 2016].
- [2] J. Morgan, "How Podcasts Have Changed in Ten Years: By the Numbers", *Medium*, 2015. [Online]. Available: <https://medium.com/@slowerdawn/how-podcasts-have-changed-in-ten-years-by-the-numbers-720a6e984e4e#.366lbn43z>. [Accessed: 29- May- 2016].
- [3] M. Massé, *REST API Design Rulebook*. Sebastopol, CA: O'Reilly, 2012.
- [4] D. Marshall, "Remote Procedure Calls (RPC)", Cardiff School of Computer Science & Informatics, 1999. [Online]. Available: <https://www.cs.cf.ac.uk/Dave/C/node33.html>. [Accessed: 29- May- 2016].
- [5] M. Ekstrand, "Tools and Experiments for Identifying Recommender Differences", Ph.D, University of Minnesota, 2014.